



KAN vs MLP

Как устроено и зачем

Почему MLP — это «чёрный ящик»?

Многослойный персептрон (MLP) — основа современного глубокого обучения:

- Фиксированные функции активации (ReLU, SiLU) в **узлах**
- Обучаются только **скалярные веса** на рёбрах
- Выразительная сила гарантирована универсальной теоремой аппроксимации

Проблема:

- MLP — это закрытый «чёрный ящик»
- Мы не можем понять, как именно сеть пришла к решению
- Это критично для научных задач, где важна интерпретируемость

Идея KAN: вывернуть MLP наизнанку

KAN основаны на **теореме Колмогорова–Арнольда** (1957):

любая непрерывная функция f от n переменных на компакте может быть представлена в виде композиции одномерных функций и операций сложения:

$$f(x) = \sum_{q=1}^{2^{n+1}} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right),$$

Ключевая идея: функции активации находятся **на рёбрах**, а не в узлах.

Параметры	MLP	KAN
Активации	В узлах (фиксированные)	На рёбрах (обучаемые)
Веса	Скалярные	Одномерные функции
Узлы	Вычисляют активации	Только суммируют

MLP vs KAN: сравнение архитектур

MLP:

- Нейроны = фиксированные функции активации
- Рёбра = скалярные веса
- Граф вычислений: активация → взвешивание → сумма

KAN:

- Нейроны = сумматоры
- Рёбра = обучаемые функции $\phi(x)$
- Граф вычислений: $x \rightarrow \phi(x) \rightarrow$ сумма
- Слой: $x_{l+1,j} = \sum_{i=1}^{n_{in}} \phi_{l,j,i}(x_{l,i})$

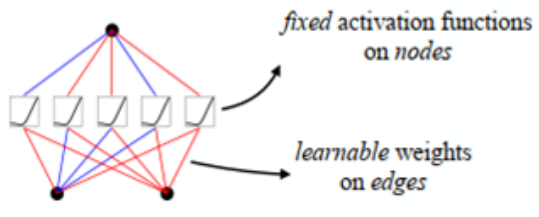
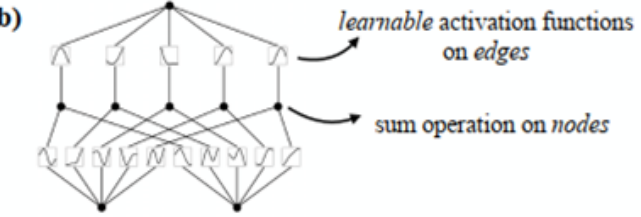
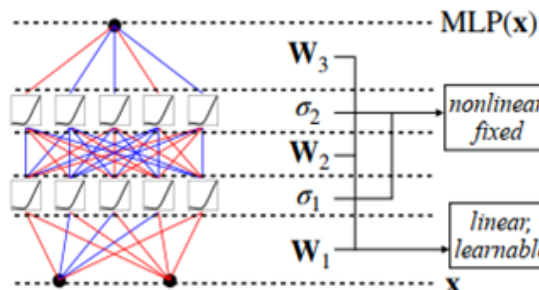
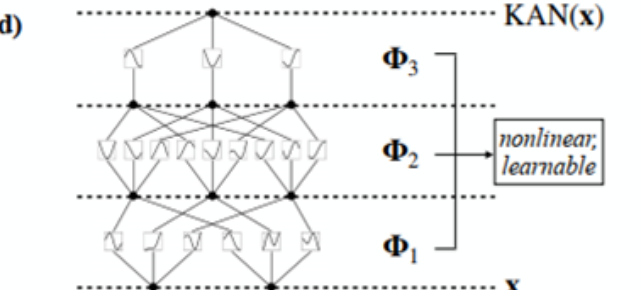
Model	Multi-Layer Perceptron (MLP)	Kolmogorov-Arnold Network (KAN)
Theorem	Universal Approximation Theorem	Kolmogorov-Arnold Representation Theorem
Formula (Shallow)	$f(\mathbf{x}) \approx \sum_{i=1}^{N(\epsilon)} a_i \sigma(\mathbf{w}_i \cdot \mathbf{x} + b_i)$	$f(\mathbf{x}) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right)$
Model (Shallow)	(a) 	(b) 
Formula (Deep)	$\text{MLP}(\mathbf{x}) = (\mathbf{W}_3 \circ \sigma_2 \circ \mathbf{W}_2 \circ \sigma_1 \circ \mathbf{W}_1)(\mathbf{x})$	$\text{KAN}(\mathbf{x}) = (\Phi_3 \circ \Phi_2 \circ \Phi_1)(\mathbf{x})$
Model (Deep)	(c) 	(d) 

Рисунок 1: Многослойные персептроны (MLP) против сетей Колмогорова-Арнольда (KAN)

Обучаемая функция на ребре

Каждая функция активации представляется как:

$$\phi(x) = \omega_b b(x) + \omega_s \text{spline}(x),$$

$$\text{spline}(x) = \sum_i c_i B_i(x)$$

Компоненты:

$b(x)$ — базисная функция (обычно $\text{silu}(x) = x \cdot \sigma(x)$);

$B_i(x)$ — В-сплайновые базисные функции порядка k (кубические, $k=3$);

c_i — обучаемые коэффициенты сплайна;

ω_b, ω_s — обучаемые масштабирующие коэффициенты.

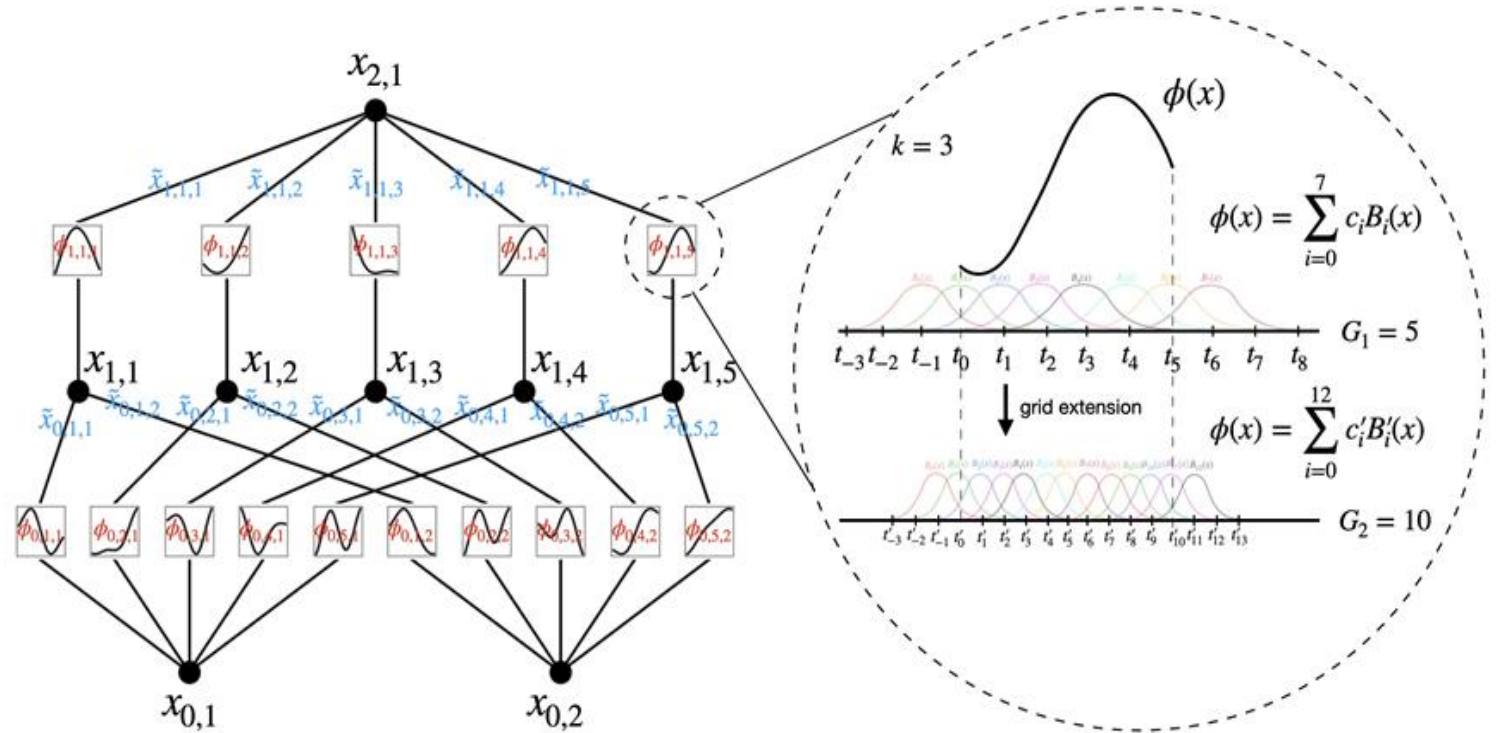


Рисунок 2: Слева: Обозначения активаций, проходящих через сеть.

Справа: функция активации параметризована как В-сплайн, что позволяет переключаться между крупнозернистыми и мелкозернистыми сетками.

Обучение KAN vs MLP — ключевые различия

Базовый процесс

	MLP	KAN
Алгоритм	Стандартный backpropagation	Backpropagation + дополнительные шаги
Оптимизаторы	SGD, Adam (стабильно)	Adam, LBFGS (при $G > 100$ LBFGS замедляется)
Градиенты	По скалярным весам w	По коэффициентам сплайнов c_i и масштабам w_b, w_s
Регуляризация	L1/L2, dropout	L1 на сплайновых коэффициентах (разреживание сети)

Специфические шаги обучения KAN

1. Динамическое обновление сетки (обязательно)

- В-сплайны определены на фиксированной сетке $[a, b]$
- Входные значения могут выходить за пределы $\rightarrow$ нестабильность
- Решение: периодическая **перестройка сетки** под текущее распределение данных

2. Расширение сетки - увеличение числа интервалов сетки $G \rightarrow G'$ для повышения точности

3. Чувствительность к гиперпараметрам

- Требуется настройка: порядок сплайна k (обычно 3), число интервалов G , частота обновления
- Шум $\rightarrow$ осцилляции сплайнов $\rightarrow$ необходима регуляризация

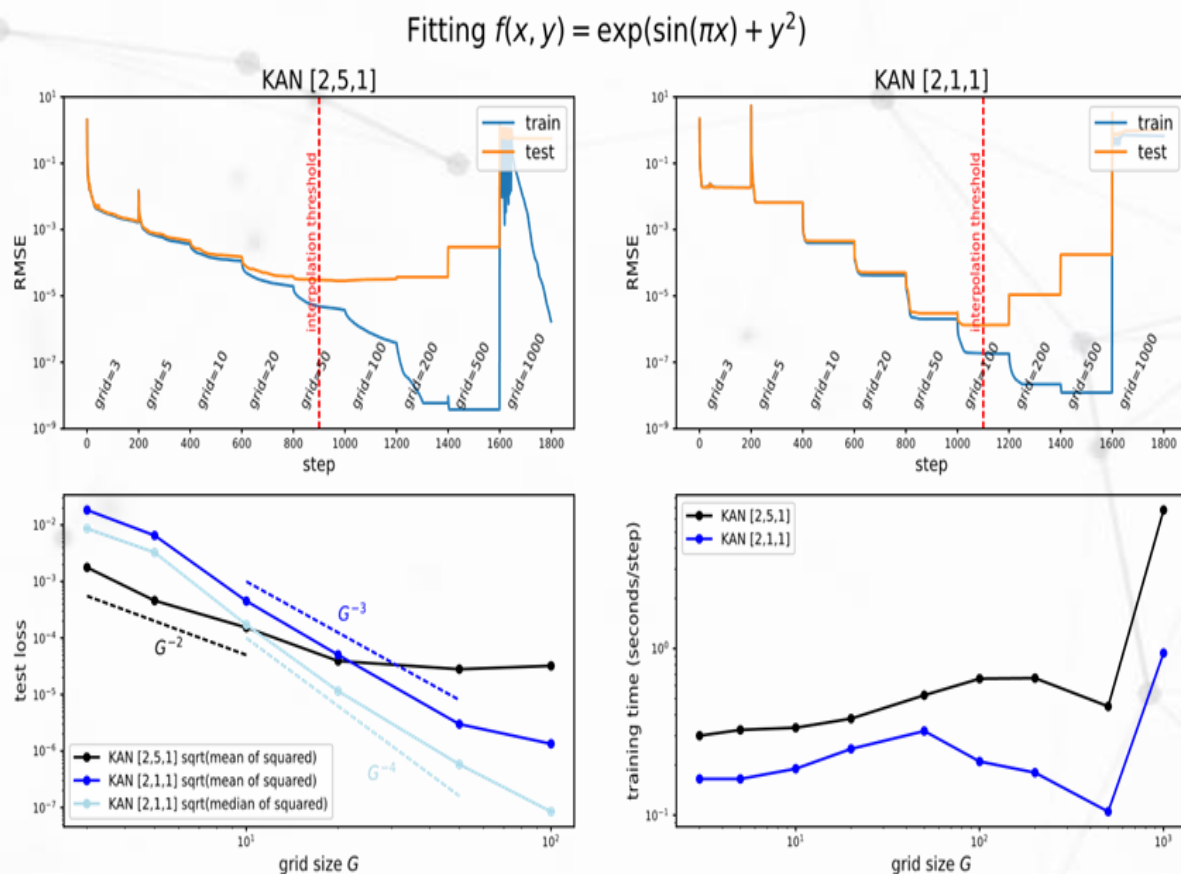
Ключевое отличие №1 — Расширение сетки

Как KAN повышает точность без переобучения

В MLP: хотите повысить точность → увеличиваете ширину/глубину → **переобучаете с нуля** (дорого)

В KAN: есть процедура Grid Extension:

1. Обучаете сеть на грубой сетке (G интервалов)
2. Строите мелкую сетку ($G' > G$ интервалов)
3. Инициализируете коэффициенты новой сетки через наименьшие квадраты
4. Дообучаете — точность растёт **без переобучения с нуля**



Теоретическая оценка: $\|f - \text{KAN}_G\| \leq C \cdot G^{-k}$
Для кубических сплайнов ($k=3$): ошибка убывает как G^{-3} — очень быстро!

Рисунок 3: Вверху: динамика обучения KAN [2,5,1] ([2,1,1]) с расширением сетки. Внизу слева: тест RMSE следует законам масштабирования в зависимости от размера сетки G . Внизу справа: время обучения благоприятно масштабируется с размером сетки G .

Ключевое отличие №2 — Интерпретируемость

KAN — это «белый ящик»

MLP даёт ответ, но не объяснение.

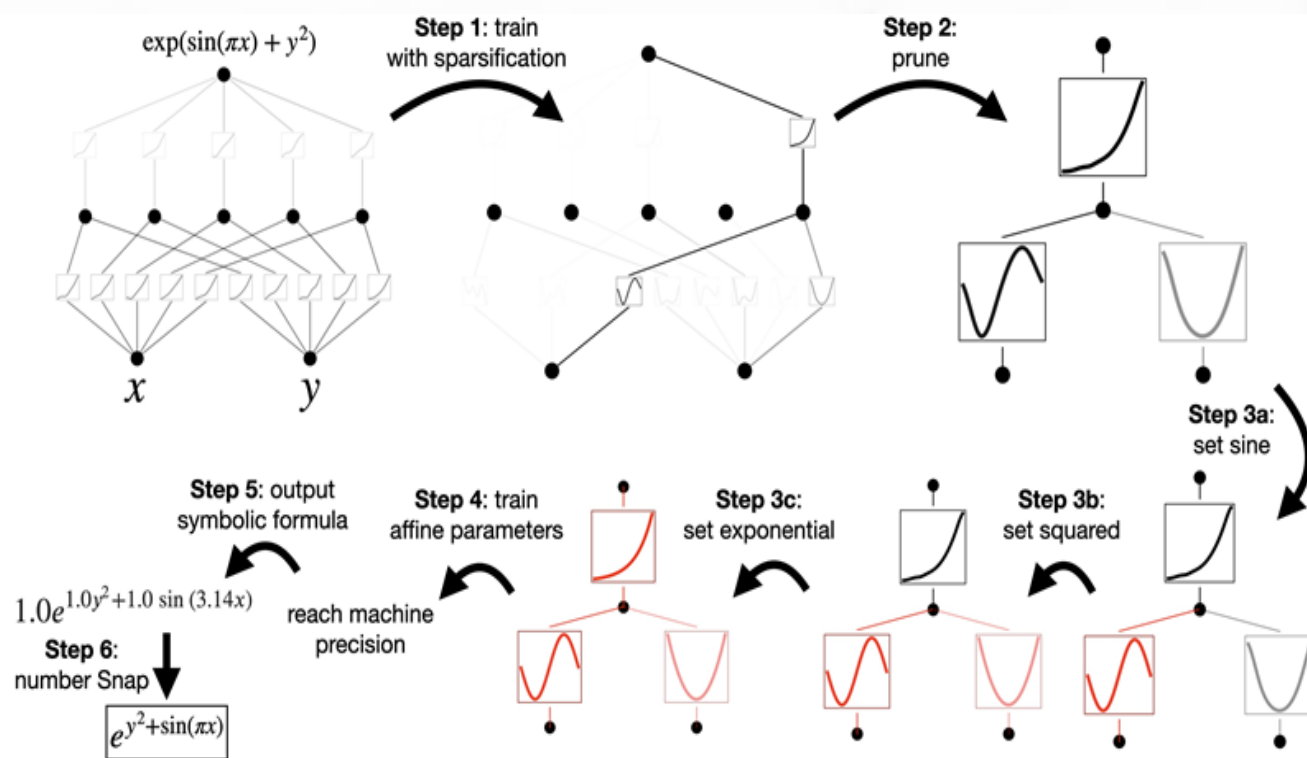


Рисунок 4: Пример символической регрессии с помощью KAN.

KAN можно:

- **Визуализировать:** каждое ребро — это функция, её можно нарисовать
- **Анализировать:** определить важность каждого признака
- **Извлекать формулы:** превратить сеть в символическое выражение

Пример: обучение на $f(x,y)=\sin(x)+ey$

- KAN выделит одно ребро для $\sin(x)$, другое для ey
- Мы увидим это в визуализации

Для науки это революция: ИИ не просто предсказывает, а **открывает законы**

Библиотека ruKan: AI + Science

Три новых компонента:

- **MultKAN** — узлы умножения
 - Выявление мультипликативных структур ($y=x \cdot y$)
 - Нет обучаемых параметров в узлах умножения
- **kanpiler** — компилятор формул
 - Превращает символьную формулу в KAN
 - Парсинг → модификация → сборка
- **Tree converter** — преобразование в деревья
 - Строит интерпретируемое дерево решений
 - Открывает лагранжианы, симметрии, законы

+ Оптимизации:

- Память: $O(N^2 LG) \rightarrow O(L \cdot N \cdot G)$
- Поддержка GPU
- Ускорение в сотни раз

Двунаправленная синергия

Science → KAN (встраивание знаний)

Уровень	Инструмент	Пользователь задает
Важные признаки	augment_input	«у зависит только от x_1 , x_2 »
Модульные структуры	module	« x_1 и x_2 аддитивно влияют на y »
Точная формула	kanpiler	« $y = \sin(x_1) + \exp(x_2)$ »

KAN → Science (извлечение знаний)

Инструмент	Что даёт
prune_input	Определяет важные признаки
auto_swap	Выявляет модули
auto_symbolic + symbolic_formula	Извлекает формулы
rewind	Проверяет гипотезы

Сравнение параметров и сложности

Нюансы:

- KAN требуют **больше параметров** при одинаковой ширине
- KAN используют **значительно меньшую ширину** ($N \approx 10$ вместо $N \approx 100$)
- Это **компенсирует** разницу

Скорость:

- KAN медленнее MLP в **10–100 раз** из-за B-сплайнов
- Сложно распараллелить

Параметр	MLP	KAN
Параметро в на слой	$O(N^2)$	$O(N^2G)$
Глубина	L	L
Итого	$O(N^2L)$	$O(N^2LG)$

Экспериментальная проверка: восстановление закона затухающего осциллятора

Физическая система:

Уравнение движения: $m\ddot{x} + c\dot{x} + kx = 0$

Отсюда ускорение: $a = -\frac{k}{m}x - \frac{c}{m}v$.

Параметры (одна среда):

$m=1.0$ кг, $k=4.0$ Н/м, $c=0.3$ Н·с/м

Истинный закон: $a=-4.0x-0.3v$

Цель эксперимента:

Сравнить точность, интерпретируемость и устойчивость к шуму; извлечь аналитическую формулу из KAN и сопоставить с MLP.

Данные:

- 9 траекторий (численное интегрирование, `solve_ivp`)
- 500 точек на траекторию, $t \in [0, 20]$
- Всего 4500 точек, разбиение 80%/20% (train/test)
- Аддитивный гауссов шум к ускорению: уровни 0%, 1%, 3%, 5%, 7%, 10% (и до 75% в отдельных тестах)

Модели:

- **KAN**: архитектура [2,2,1], линейные B-сплайны ($k=1$), `grid=10`, ~66 параметров
- **MLP**: $2 \rightarrow 16 \rightarrow 1$, активация ReLU, 65 параметров (сравнимое число)

Сравнение KAN и MLP (фиксированная среда, 1% шума)

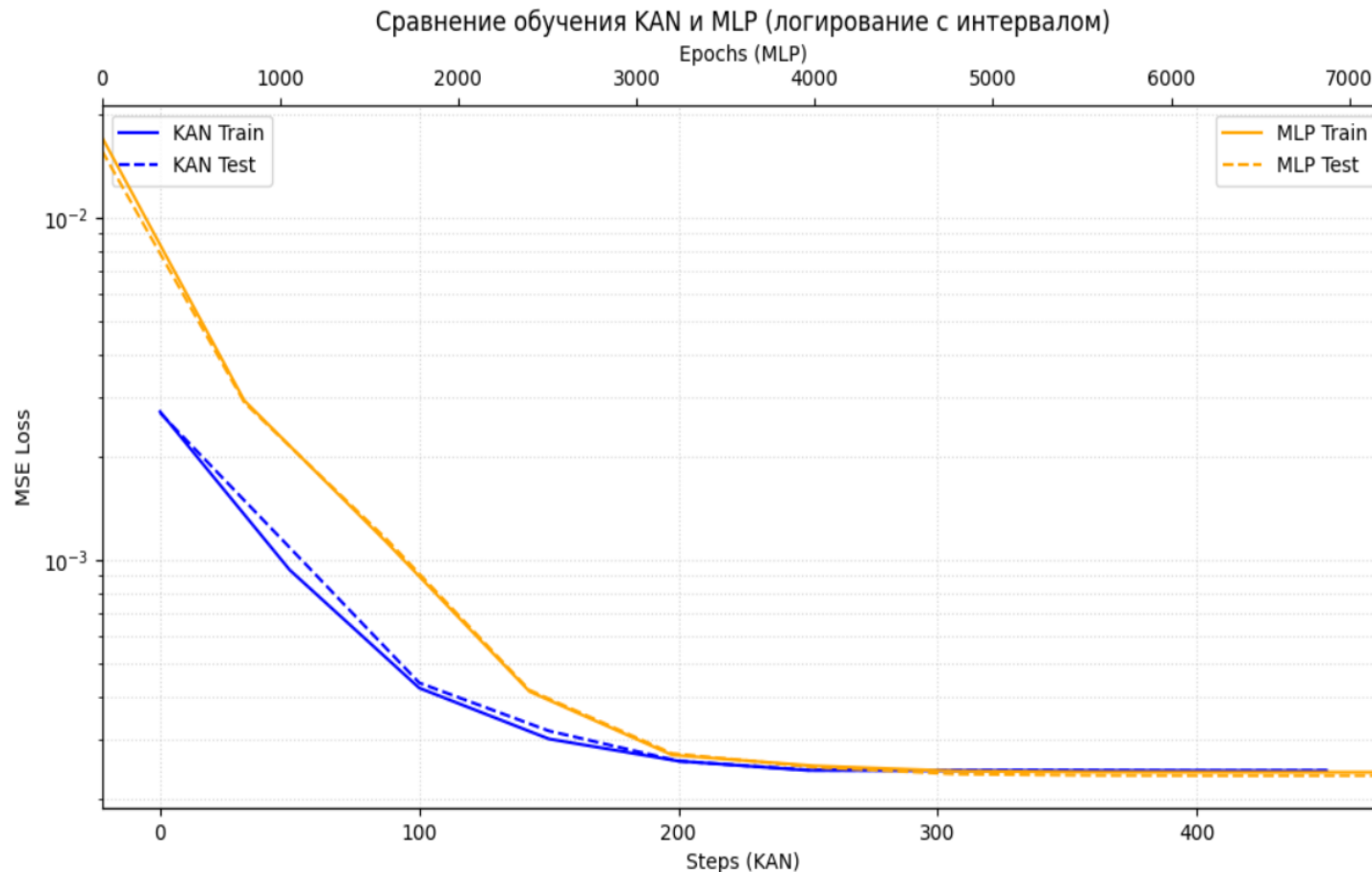
Точность почти одинакова, но KAN интерпретируема

Качество предсказания (MSE на тесте):

- KAN = $2.43 \cdot 10^{-4}$,
- MLP = $2.35 \cdot 10^{-4}$

Скорость сходимости:

- KAN (L-BFGS) выходит на плато за ~300 шагов
 - MLP (Adam) требует >5000 эпох
- KAN эффективнее по итерациям, но шаг L-BFGS тяжелее



Сравнение KAN и MLP (фиксированная среда, 1% шума)

Точность почти одинакова, но KAN интерпретируема

Визуализация архитектуры KAN:

- До обучения: все рёбра равномерны
- После обрезки: остаются только два значимых ребра ($x \rightarrow a$ и $v \rightarrow a$), толщина пропорциональна вкладу

Символьная формула, извлечённая из KAN:

$$a = -4.0841 \cdot x - 0.3008 \cdot v - 0.0009$$

Отклонение от истины < 2.1%

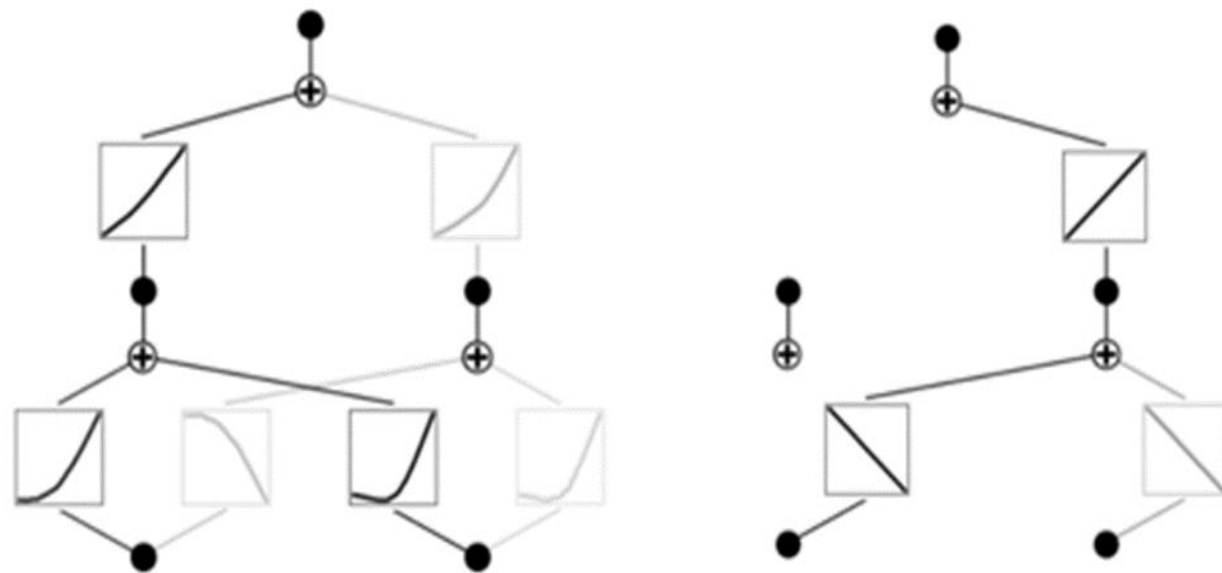


Рисунок 5 KAN до обучения (слева) и обрезанный KAN после обучения (справа).

Вывод: MLP – чёрный ящик (только ответ). KAN – белый ящик (ответ + формула + визуализация).

Влияние уровня шума (0%...10%)

Коэффициенты устойчивы, но интерпретируемость падает

Сравнение MSE и численных коэффициентов KAN:

Шум	MSE KAN	MSE MLP	α (коэф. при x)	β (коэф. при v)
0%	$5.5 \cdot 10^{-8}$	$6.0 \cdot 10^{-6}$	-4.0001	-0.3002
1%	$2.3 \cdot 10^{-4}$	$2.3 \cdot 10^{-4}$	-3.9983	-0.3037
3%	$2.0 \cdot 10^{-3}$	$2.0 \cdot 10^{-3}$	-4.0117	-0.3001
5%	$5.6 \cdot 10^{-3}$	$5.5 \cdot 10^{-3}$	-4.0089	-0.2981
7%	$1.1 \cdot 10^{-2}$	$1.1 \cdot 10^{-2}$	-4.0135	-0.3137
10%	$2.2 \cdot 10^{-2}$	$2.2 \cdot 10^{-2}$	-4.0238	-0.3087

Вывод: Численные коэффициенты надёжны, но визуальная интерпретируемость и простота символьной формулы ухудшаются с ростом шума.

Наблюдения:

- MSE обеих моделей практически одинаковы при всех шумах
- Коэффициенты α и β остаются в пределах ± 0.02 от истинных (-4.0 и -0.3) даже при 10% шума

Визуализация архитектуры KAN (вставить три рисунка: при 0%, 3%, 7%):

- 0% – максимальная разреженность (только два ребра)
- 3% – появляются дополнительные тонкие связи
- 7% – структура теряет разреженность, сеть пытается учесть шум

Символьная регрессия: «сырые» формулы vs ограничение библиотеки

Автоматическая регрессия чувствительна к шуму

Примеры формул, выданных auto_symbolic:

Шум	Символьная формула KAN (auto_symbolic)
0%	$31.555 - 35.5505 \cdot \exp(0.0001 \cdot \exp(4.2 \cdot x_2)) - 0.1188 \cdot \exp(-9.792 \cdot x_1)$
3%	$26.6992 - 26.5359 \cdot \exp(0.0048 \cdot \exp(3.4001 \cdot x_1)) + 0.0021 \cdot \exp(1.8 \cdot x_2)$
7%	$-0.39 \cdot x_1 - 97.8607 \cdot \exp(0.0071 \cdot \exp(1.8081 \cdot x_1)) + 0.0001 \cdot \exp(3.4282 \cdot x_2) + 98.5487 + 0.0135 \cdot \exp(-5.8 \cdot x_2)$

Вывод: Для высокой интерпретируемости автоматическую символьную регрессию нужно дополнять обрезкой, дообучением с замороженной сеткой или сужением библиотеки.

Проблема: даже при 0% шума появляются экспоненциальные «хвосты», при $\geq 3\%$ формула становится нечитаемой физически.

Решение – принудительно линейная библиотека (только 'x'):

- При любом шуме (вплоть до 75%) получаем чистую линейную формулу $a = \alpha x + \beta v + \text{const}$
- Пример при 10%: $a = -4.003x - 0.226v - 0.0374$ (коэффициенты близки к истинным)

Ограничение: требует априорного знания о линейности закона. В общем случае нужна постобработка или ручная регрессия.

Обобщение без инженерии признаков (MultKAN)

Автоматическое открытие мультипликативной структуры?

Идея: Подать на вход только 5 исходных переменных: $(x, v, k, c, 1/m)$ – без производных комбинаций. Использовать MultKAN с узлами умножения, чтобы сеть сама «открыла» произведения $(k/m)x$ и $(c/m)v$.

Эксперимент 1 – дискретные параметры

- $k \in \{2, 4, 6\}$, $c \in \{0.1, 0.3, 0.5\}$, $m \in \{0.5, 1.0, 2.0\}$
- Архитектура $[5, [4, 2], 1]$ с кубическими сплайнами
- Результат: $MSE \sim 2.0$, предсказания хаотичны

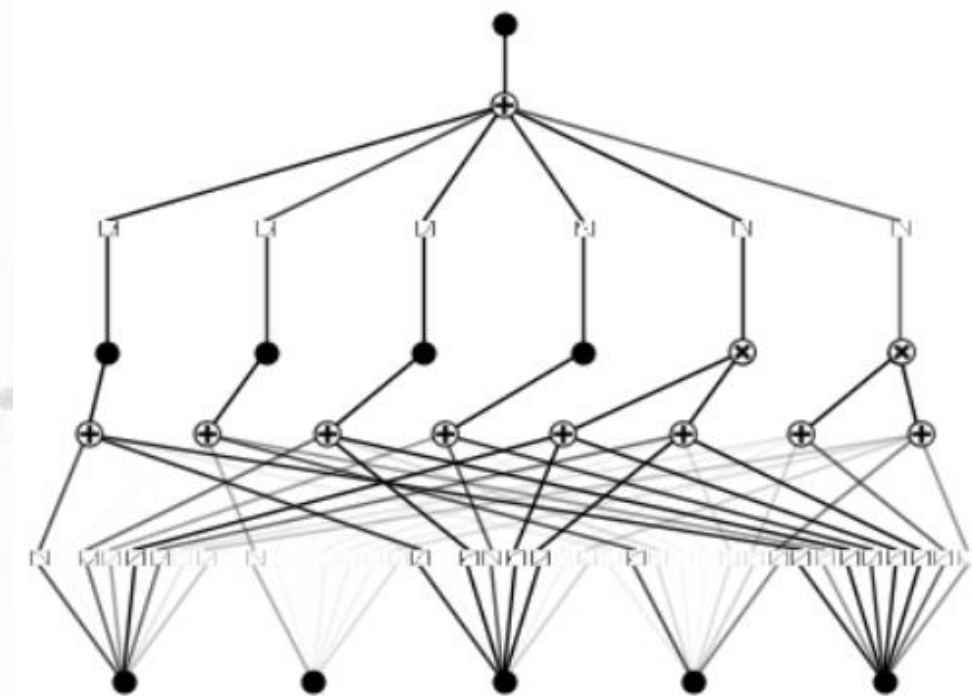


Рисунок 6 Визуализация итоговой архитектуры

Обобщение без инженерии признаков (MultKAN)

Автоматическое открытие мультипликативной структуры!

Эксперимент 2 – почти непрерывные параметры

- Диапазоны: $k \in [1, 8]$, $c \in [0.05, 0.8]$, $m \in [0.3, 2.5]$ (по 7 значений)
- Данных 514 500 точек, архитектура $[5, [4, 2], 1]$ с линейными сплайнами
- MSE на тесте снизилась до $4.1 \cdot 10^{-3}$ – численно модель работает

Однако:

- символьная регрессия выдаёт сложные нелинейные выражения
- линейная регрессия предсказаний на правильные физические комбинации $(k/m)x$ и $(c/m)v$ даёт $R^2=0.40$ – структура не выделена

Вывод: MultKAN не способен самостоятельно сформировать простую мультипликативную формулу. Для получения интерпретируемого закона необходима инженерия признаков.

Обобщение с инженерией признаков (11 входов)

Правильные признаки $\rightarrow$ идеальное восстановление закона

Расширенный набор признаков (11 входов):

$x, v, k, c, m, k/m, c/m, kx, cv, kx/m, cv/m$
(масштабированы только x и v)

Архитектура:

KAN [11, 2, 1], линейные B-сплайны ($k=1$, $\text{grid}=3$)
Обучение: Adam (500 шагов) + L-BFGS (300 шагов)
+ обрезка (два этапа)

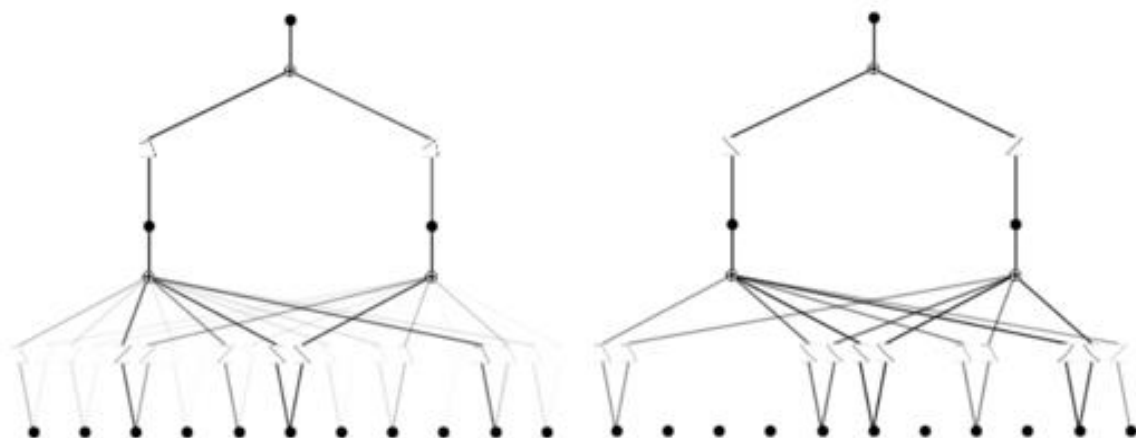


Рисунок 6 Начальная и конечная структура обобщенной модели

Результат после обрезки:

- Сеть сохранила только **два** значимых ребра: от kx/m и cv/m
- Символьная регрессия (библиотека 'x') даёт: $a = -0.9990 \cdot (k \cdot x/m) - 0.9986 \cdot (c \cdot v/m)$ + (пренебрежимо малые члены).
- MSE на тесте: $6 \cdot 10^{-6}$ (на несколько порядков ниже, чем без инженерии)

Вывод: Симбиоз предметных знаний и KAN даёт полностью интерпретируемую, точную и обобщаемую модель – идеальный инструмент для открытия законов.

Выводы по экспериментам

KAN – мощный нишевый инструмент для AI+Science

- KAN успешно восстанавливает физический закон, выдавая символьную формулу (в отличие от MLP).
- Численные коэффициенты устойчивы к шуму (до 10% и выше).
- «Сырая» символьная регрессия чувствительна к шуму; для чистоты нужна постобработка или ограничение библиотеки.
- Обобщение без инженерии признаков (MultKAN) — практически нереализуемо; сеть не выделяет мультипликативную структуру.
- При разумном инжиниринге признаков даже простейшая KAN даёт идеальный результат.
- KAN требует ручной настройки (grid, порядок сплайна, обрезка) и медленнее MLP, но для научных задач его интерпретируемость перевешивает.

Итог: KAN — это «белый ящик» для AI+Science, но его применение должно сопровождаться предметными знаниями.

Где KAN побеждает MLP

Области превосходства

- ✓ **Символьная регрессия** - восстановление математических формул из данных
- ✓ **Научные вычисления (AI+Science)** - физика, химия, биология, открытие законов, сохраняющихся величин
- ✓ **Решение дифференциальных уравнений (PDE)** - более быстрые законы масштабирования
- ✓ **Табличные данные с небольшим числом признаков** - превосходная точность при малом размере сети
- ✓ **Интерпретируемые модели** - возможность объяснить решение

Где MLP остаётся лучше

Ограничения KAN

- ✗ **Крупные сенсорные данные** (изображения, видео, аудио) MLP / CNN / Transformer лучше
- ✗ **Обработка естественного языка (NLP)** - тексты — неструктурированные данные
- ✗ **Классическое машинное обучение с большим числом признаков** - MLP проще масштабировать
- ✗ **Индустриальные приложения общего назначения** - MLP быстрее, дешевле, проще в развёртывании
- ✗ **Глубокие сети ($L > 5$)** - KAN не стабильно превосходят MLP, проблемы с осцилляциями сплайнов

Подведение итогов

Зачем нужен KAN?

KAN — это мощный нишевый инструмент для:

Научных исследований

- Интерпретируемость
- Открытие законов
- Извлечение формул

Задач со структурными данными

- Гладкие композиционные функции
- Мало признаков
- Требуется точность

KAN – НЕ замена MLP

**Для промышленных задач
общего назначения → MLP
остаётся выбором №1**

**Эксперименты подтверждают, KAN
— это интерпретируемая
альтернатива MLP для научных
задач, но требует осмысленного
симбиоза с предметной областью.**

AI+Science:

- Симбиоз ИИ и науки
- Двусторонний обмен знаниями